



Politechnika Wrocławska

Projekt Tripterona

Bartłomiej Łuczeńczyk 277592

Mikołaj Mroczko 277576

Paweł Czerny 282945

Kostiantyn Ivanov 277567

Spis treści

1	Koncept	3
2	Wymagania	4
3	Architektura	5
4	Schemat blokowy architektury	6
5	Mechanika	7
5.1	Model 3D konstrukcji	7
5.2	Pole robocze	7
5.3	Symulacja	9
5.4	Problemy konstrukcyjne	9
5.5	Interfejs mechanika–elektronika	10
5.6	Spis elementów konstrukcyjnych	11
6	Elektronika	12
6.1	Schemat ideowy	12
6.2	Model płytki	13
6.3	Zastosowane moduły	14
6.4	Zasilanie	14
6.5	Spis elementów elektronicznych	15
7	Oprogramowanie	16
7.1	Architektura sterowania	16
7.2	Konfiguracja sprzętowa układu	16
7.3	Procedura Homing	18
7.4	Trajektoria robocza	19
7.5	Dodatkowe funkcje programu	21
7.5.1	Definicja maszyny stanów	21
7.5.2	Logika START	21
7.5.3	Pozycja startowa	22
7.5.4	Obsługa STOP	22
8	Załączniki	23

1 Koncept

Projekt tripterona stanowi alternatywną formę konstrukcji mechanizmu równoległego, który zachowuje równoległość efektora względem płaszczyzny oraz umożliwia niezależny ruch w poszczególnych osiach. Konstrukcja została zaprojektowana z myślą o uniwersalności i łatwej adaptacji do różnych zadań realizowanych przez koło naukowe EPIC.

Platforma ta będzie wykorzystywana do automatyzacji procesu wykonywania zgłędów metalograficznych. Ze względu na ograniczony budżet koła naukowego zakup profesjonalnych urządzeń oraz dedykowanych narzędzi nie jest możliwy, dlatego zaproponowane rozwiązanie alternatywne spotkało się z pozytywnym odbiorem zarządu koła.

Aby zminimalizować koszty produkcji urządzenia, podjęto decyzję o wykorzystaniu profili aluminiowych oraz elementów wytwarzanych metodą druku 3D, uzupełnionych o niezbędne komponenty elektroniczne, sensory i akulatory.

Należy zaznaczyć, że przedstawiona konstrukcja stanowi jedynie prototyp urządzenia. Wersja końcowa może istotnie różnić się pod względem gabarytów, dokładności wykonania oraz zastosowanych materiałów.

Możliwy jest również dalszy rozwój projektu oraz opracowanie nowych, dedykowanych wersji urządzenia, dostosowanych do konkretnych zastosowań.

2 Wymagania

Wymagania interesariuszy

- Umożliwia precyzyjne pozycjonowanie efektora w przestrzeni roboczej,
- zapewnia ruch niezależny w trzech osiach (X, Y, Z),
- zachowuje równoległość efektora względem płaszczyzny roboczej,
- rozwiązanie niskokosztowe,
- łatwy w transporcie i adaptacji do różnych zastosowań,

Wymagania systemowe

- Układ ruchu oparty o mechanizm równoległy,
- odpowiedni obszar roboczy (architektura),
- umożliwia sterowanie poprzez mikrokontroler,
- umożliwia integrację odpowiednich końcówek roboczych.

Wymagania funkcjonalne

- przemieszczanie efektora w osiach X, Y, Z,
- utrzymywanie orientacji efektora,
- wykonywanie zaprogramowanej trajektorii ruchu,

Wymagania niefunkcjonalne

- wykorzystanie łatwo dostępnych komponentów,
 - łatwość montażu i demontażu, powtarzalności działania urządzenia,
 - możliwość przenoszenia urządzenia,
-

3 Architektura

Warstwa mechaniczna

- profile aluminiowe,
- elementy łączące (łączniki kątowe, łączniki systemowe),
- ramiona mechanizmu wykonane metodą druku 3D,
- wózki liniowe,
- prowadnice liniowe,
- platforma efektora,
- koła pasowe, napinacze, taśmy napędowe,
- mocowania silników, obudowy elementów.

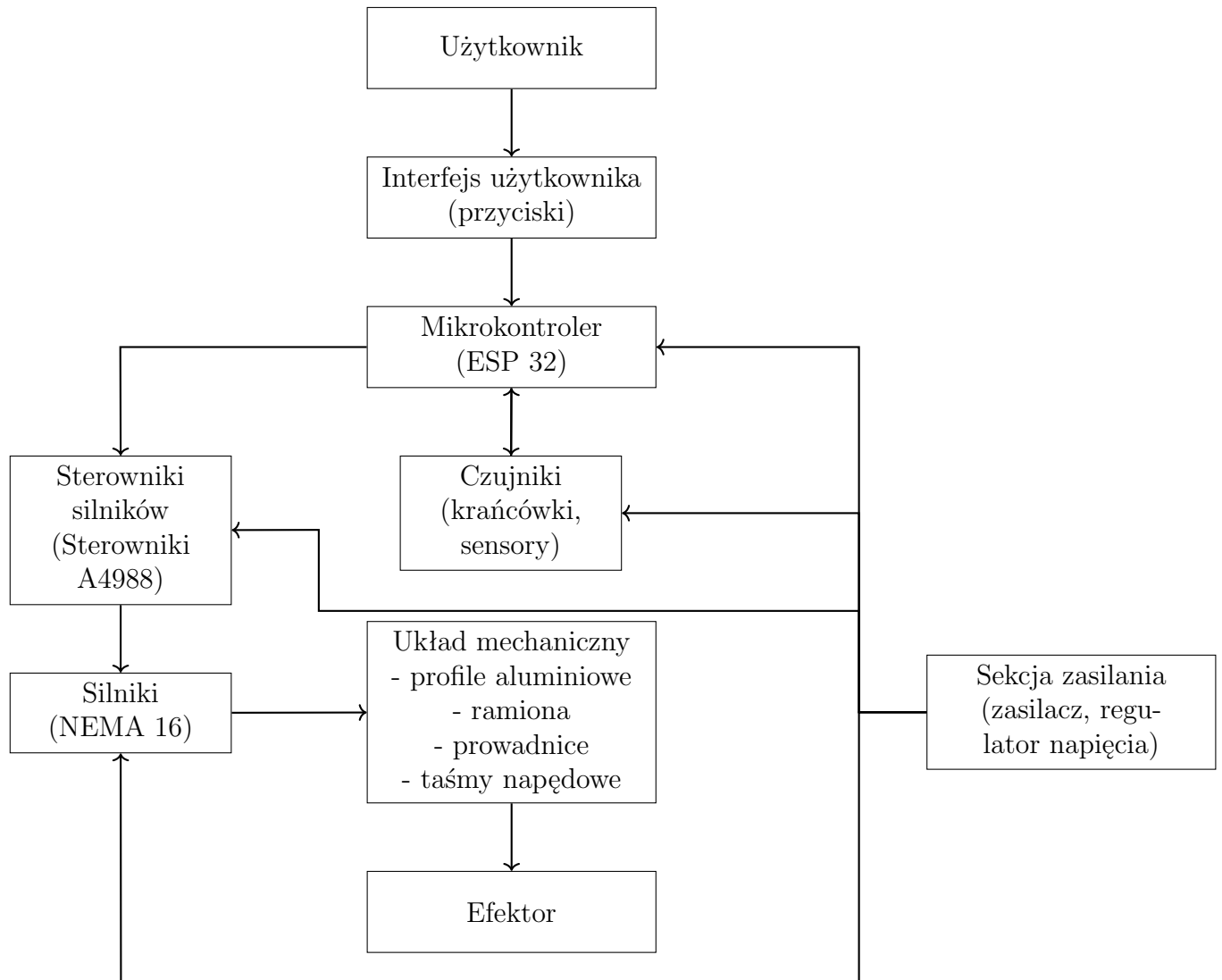
Warstwa elektroniczna

- mikrokontroler ESP 32,
- płyta główna,
- sterownik silników krokowych,
- silniki krokowe NEMA 16,
- czujniki krańcowe,
- zasilacz,
- Przyciski start/stop.

Warstwa programowa

- sterowanie silnikami krokowymi,
 - obsługa czujników krańcowych,
 - procedura homing,
 - kontrola trajektorii ruchu,
-

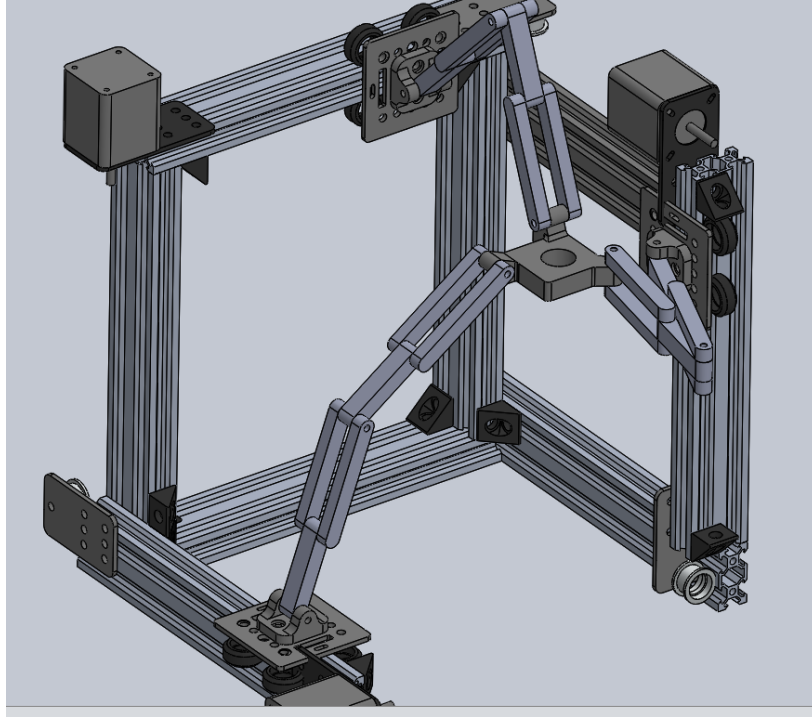
4 Schemat blokowy architektury



Rysunek 1: Schemat blokowy architektury systemu tripterona

5 Mechanika

5.1 Model 3D konstrukcji



Rysunek 2: Render modelu 3D zaprojektowanego układu

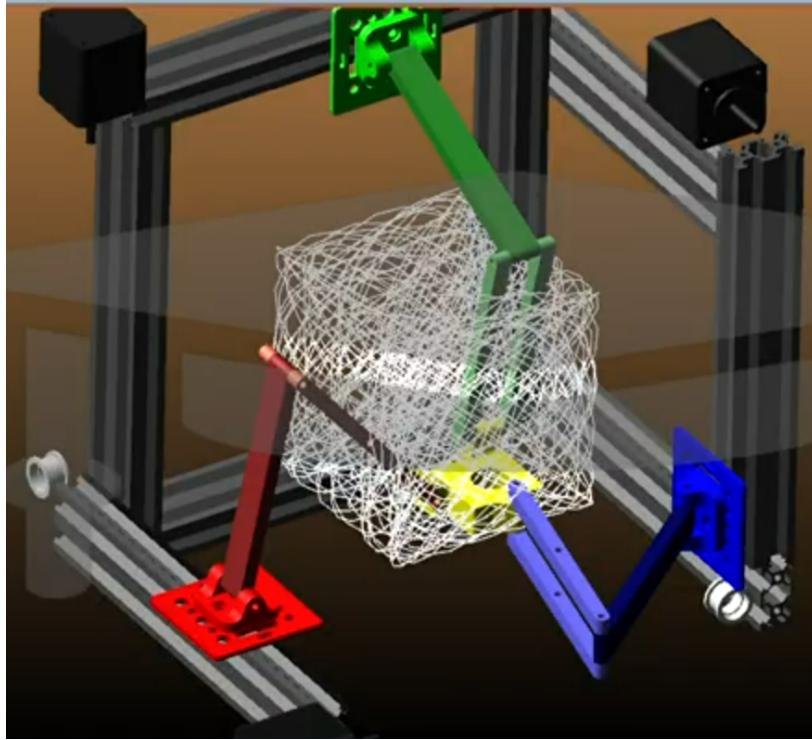
Rysunek przedstawia finalną geometrię układu po etapach iteracji projektowej. Model stanowi podstawę do analizy mechanicznej i przeprowadzonych symulacji.

Konstrukcja wykonana została ze standardowych profili aluminiowych oraz elementów wykonanych w technice druku 3D. Zastosowanie profili aluminiowych typu V-Slot zapewnia odpowiednią sztywność całej struktury przy jednoczesnym zachowaniu stosunkowo niskiej masy układu oraz umożliwia prowadzenie wózków liniowych. Elementy wykonane w technologii druku 3D znacząco obniżają koszt wykonania oraz pozwalają na dostosowanie części do indywidualnych potrzeb konstrukcji, co umożliwiło szybkie iteracje projektowe oraz optymalizację kształtów pod kątem funkcjonalnym.

5.2 Pole robocze

Pole robocze zaprojektowanego układu zostało wyznaczone metodą eksperymentalną poprzez symulację ruchu końcówki efektora z wykorzystaniem przebiegów sinusoidalnych w osiach X, Y oraz Z. Takie podejście pozwoliło na odwzorowanie skrajnych

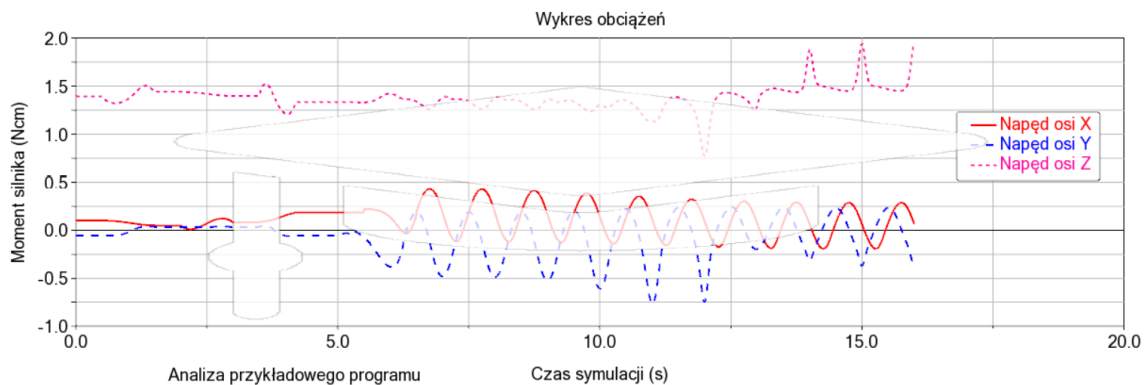
położenia mechanizmu oraz określenie rzeczywistego zakresu pracy układu. Zaprojektowany system posiada pole robocze o wymiarach $12 \times 12 \times 12$ cm, co zapewnia wystarczającą przestrzeń do realizacji zakładanych zadań oraz testów funkcjonalnych. Uzyskany zakres ruchu uwzględnia ograniczenia wynikające z geometrii ramion oraz długości elementów wykonawczych.



Rysunek 3: Wizualizacja pola roboczego flanszy efektora

Na rysunku przedstawiono wizualizację wyznaczonego pola roboczego wraz z zaznaczonym obszarem dostępnych pozycji efektora. W praktyce rzeczywisty obszar roboczy może nieznacznie odbiegać od wartości teoretycznych ze względu na luzy mechaniczne oraz elastyczność elementów konstrukcyjnych.

5.3 Symulacja



Rysunek 4: Zarejestrowane siły

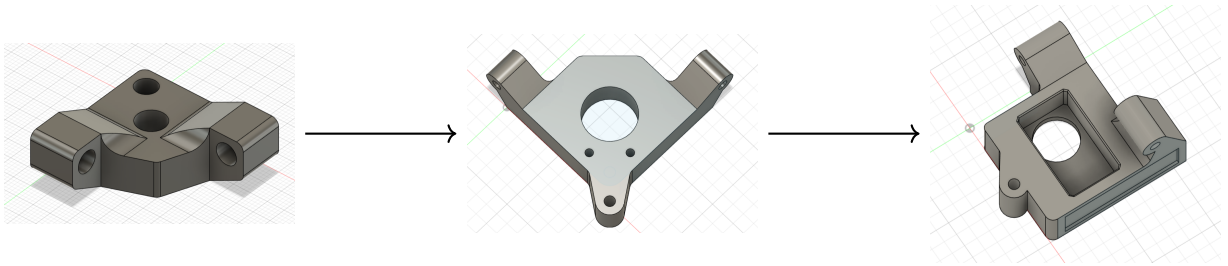
Po zamodelowaniu mechanizmu przeprowadzono symulację ruchu w programie Adams w celu wyznaczenia sił działających w poszczególnych osiach układu. Wartości zarejestrowanych sił wynosiły maksymalnie:

- 0,5 Ncm dla osi X
- 0,6 Ncm dla osi Y
- 2,0 Ncm dla osi Z

Warto zaznaczyć, że uzyskane wartości sił mogą ulegać zmianie w zależności od zastosowanego narzędzia roboczego oraz przyjętej trajektorii ruchu manipulatora. Z tego względu przy doborze silników konieczne jest uwzględnienie odpowiedniego zapasu momentu obrotowego. Niewielkie wartości wyznaczonych obciążeń zapewniły jednak dużą swobodę w projektowaniu układu napędowego.

5.4 Problemy konstrukcyjne

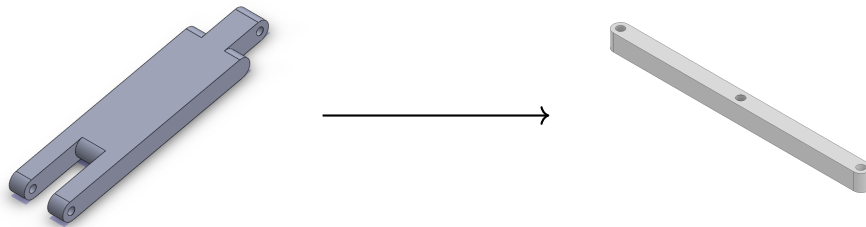
Jednym z głównych wyzwań projektowych było opracowanie flanszy montażowej efektora. Konstrukcja musiała jednocześnie nie ograniczać pola roboczego urządzenia, zapewniać możliwość łatwego montażu różnych końcówek roboczych oraz umożliwiać integrację serwomechanizmów odpowiedzialnych za sterowanie efekтором. Wymagało to kilku iteracji projektowych.



Rysunek 5: Kolejne wersje projektu flanszy efektora

Końcowa wersja pozwoliła uzyskać maksymalny obszar roboczy zostawiając pole na montaż zarówno efektorów jak i przykręcenie serwomechanizmów.

Kolejnym istotnym problemem projektowym była konstrukcja ramion robota. Głównym celem było zminimalizowanie masy całego układu przy jednoczesnym zachowaniu odpowiedniej sztywności konstrukcji. Dodatkowym wyzwaniem było takie zaprojektowanie geometrii ramion, aby uniknąć ich wzajemnego krzyżowania się w pełnym zakresie ruchu.



Rysunek 6: Kolejne wersje projektu ramion robota

5.5 Interfejs mechanika–elektronika

Na podstawie wykonanej konstrukcji oraz przeprowadzonych symulacji dobrano silniki napędowe typu Nema 16 Bipolar o momencie trzymającym 12 Ncm oraz kącie kroku $0,9^\circ$. Zastosowane silniki zapewniają odpowiedni zapas momentu względem obciążeń wynikających z masy elementów ruchomych oraz oporów mechanicznych układu. Wybór silników o mniejszym kącie kroku umożliwia realizację ruchów o wysokiej rozdzielczości, co bezpośrednio przekłada się na precyzję pozycjonowania osi roboczych.

Przyjęta konstrukcja mechaniczna pozwoliła również na określenie wymagań dla układu elektronicznego odpowiedzialnego za sterowanie ruchem. Liczba osi roboczych oraz zastosowane silniki krokowe determinują konieczność zastosowania wielokanałowego systemu sterowania umożliwiającego niezależną kontrolę każdej osi.

Ze względu na charakterystykę zastosowanych silników wymagane jest sterowanie mikrokrokowe, które zwiększa efektywną rozdzielczość ruchu oraz ogranicza drgania układu mechanicznego. Układ sterowania powinien zapewniać odpowiedni prąd fazowy, a także umożliwiać płynną regulację prędkości i przyspieszeń, co pozwala na uniknięcie gubienia kroków podczas pracy dynamicznej.

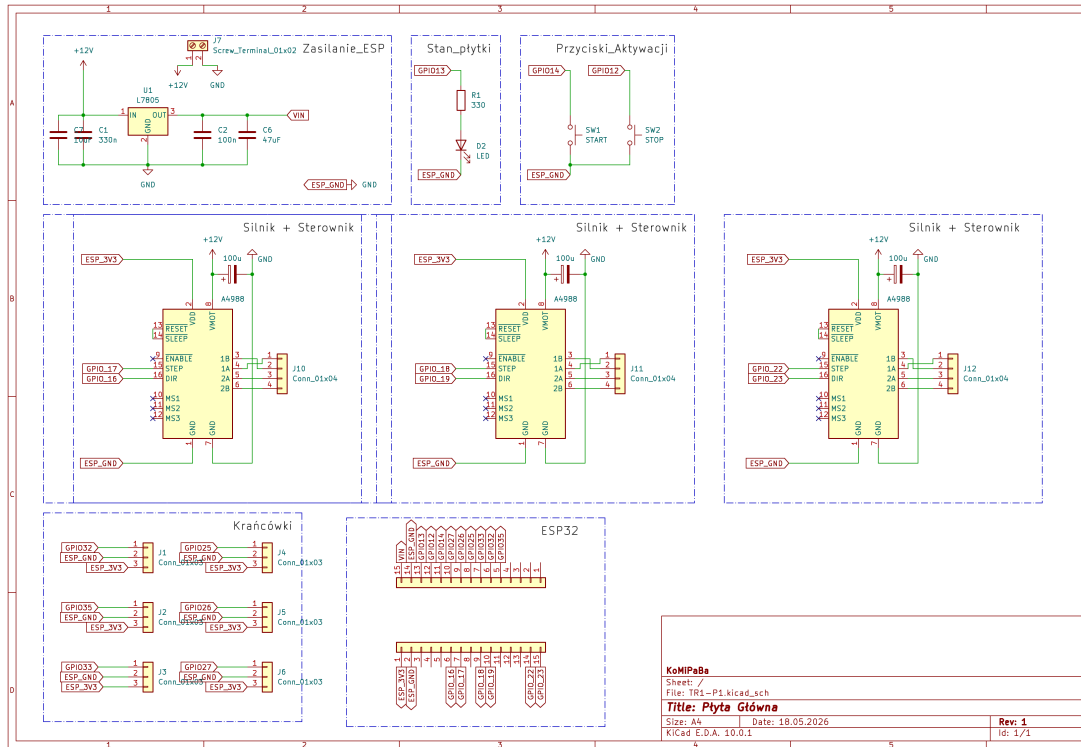
5.6 Spis elementów konstrukcyjnych

Lp.	Nazwa elementu	Kod / oznaczenie	Ilość
1	Śruba Low Profile M5x25mm	M5x25mm	12
2	Śruba Low Profile M5x8mm	M5x8mm	12
3	Dystans aluminiowy 6mm	HW1018NC	6
4	Dystans aluminiowy 3mm	HW1016NC	3
5	Double T-NUT M5	HW1029NC	6
6	T-nut	-	3
7	Zębatka GT2 (30 zębów, otw. Fi.5)	HW1036WC	3
8	Kółka (Xtreme Solid lub Solid)	HW1215WK / HW1212WK	12
9	Idler Pulley	HW1093WK	3
10	Napiacz	Druk własny	3
11	Podkładka do silnika (DYNAMIXEL)	Druk własny	3
12	Podkładka do silnika (NEMA)	Druk własny	3
13	Płyta V-SLOT Gantry Plate (Opcja 20mm)	Druk własny	3
14	Pasek GT2 5mm	HW1037GC	3
15	Profil V-SLOT 2040	2040-sr-mb	8
16	Silnik krokowy NEMA 16 Bipolar	16HM12-0634D	3
17	Eccentric Spacer 6mm	HW1073NC	6
18	Zacisk do pasków zębatych	HW1024GC	3
19	Złącze meblowe śrubowe	-	3
20	Ramię	Druk własny	3
21	Przedramię	Druk własny	6
22	Kiść	Druk własny	1
23	Bark	Druk własny	3
24	Łączniki profili	Druk własny	10

Tabela 1: Zestawienie elementów konstrukcyjnych

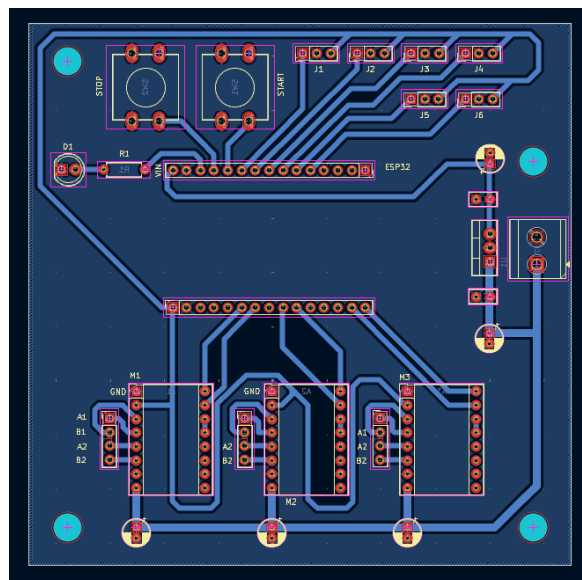
6 Elektronika

6.1 Schemat ideowy

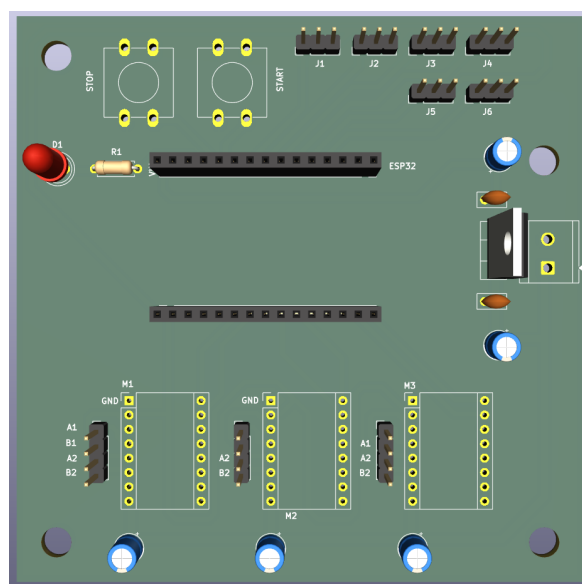


Rysunek 7: Schemat ideowy płyty głównej

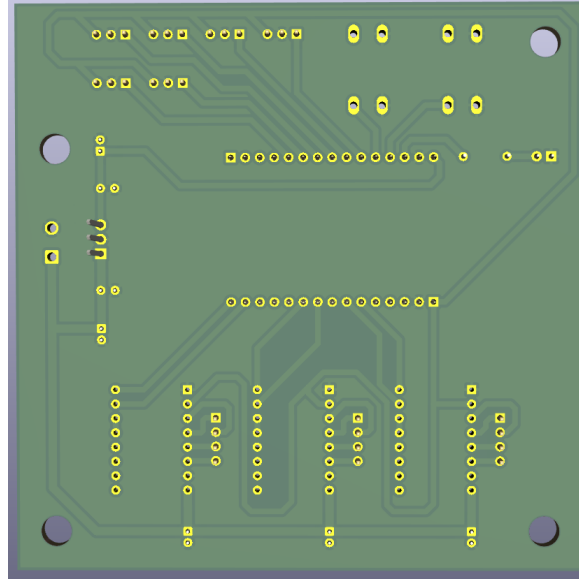
6.2 Model płytki



Rysunek 8: Schenat połączeń



Rysunek 9: Model 3D płytki



Rysunek 10: Model 3D płytki

6.3 Zastosowane moduły

Układ sterowania został oparty o mikrokontroler ESP32-WROOM, pełniący rolę jednostki nadrzędnej systemu. Mikrokontroler odpowiada za generowanie sygnałów sterujących dla sterowników silników krokowych, obsługę krańcówek oraz komunikację z użytkownikiem.

Do sterowania napędami zastosowano trzy moduły A4988, umożliwiające sterowanie bipolarnymi silnikami krokowymi w trybie pełnego kroku, półkroku oraz mikrokroku. Sterowniki realizują kontrolę kierunku obrotów (DIR) oraz liczby wykonywanych kroków (STEP).

System wyposażono w sześć krańcówek mechanicznych kompatybilnych z systemem sterowania. Ich zadaniem jest wyznaczanie pozycji zerowych manipulatora oraz zabezpieczenie układu przed przekroczeniem dopuszczalnego zakresu ruchu.

Na płytce przewidziano również diodę LED sygnalizującą stan zasilania oraz przyciski umożliwiające reset lub uruchomienie trybu pracy mikrokontrolera.

6.4 Zasilanie

Układ zasilany jest z zewnętrznego zasilacza sieciowego o napięciu 12 (lub 9) V DC, podłączanego do złącza śrubowego na płytce głównej.

Napięcie 12 V wykorzystywane jest bezpośrednio do zasilania sterowników silników krokowych A4988, które następnie dostarczają odpowiednią energię do silników krokowych.

Dla części logicznej układu zastosowano stabilizator liniowy LM7805, który obniża napięcie wejściowe 12 V do poziomu 5 V. Napięcie to jest podawane na pin VIN mikrokontrolera ESP32, gdzie następnie jest ono dalej przetwarzane przez wewnętrzny stabilizator na napięcie 3,3 V wymagane do pracy rdzenia mikrokontrolera.

Zastosowanie kondensatorów filtrujących pozwala na stabilizację napięcia oraz redukcję zakłóceń powstających podczas pracy silników krokowych, które generują dynamiczne zmiany obciążenia zasilania.

6.5 Spis elementów elektronicznych

Lp.	Nazwa elementu	Kod / oznaczenie	Ilość
1	Moduł mikrokontrolera ESP32 WROOM	ESP32-WROOM	1
2	Sterownik silnika krokowego	A4988	3
3	Złącze GoldPIN 1x15	1x15	2
4	Złącze GoldPIN 1x8	1x8	6
5	Złącze GoldPIN 1x3	1x3	6
6	Złącze GoldPIN 1x4	1x4	3
7	Przycisk THT 12x12 mm	12x12	2
8	Dioda LED	LED	1
9	Kondensator ceramiczny 100 nF	100nF	2
10	Kondensator elektrolityczny 100 μ F	100uF	3
11	Kondensator elektrolityczny 10 μ F	10uF	1
12	Kondensator elektrolityczny 47 μ F	47uF	1
13	Stabilizator napięcia	LM7805	1
14	Złącze śrubowe 1x2	1x2	1
15	Rezystor 330 Ω	330R	1
16	Krańcówka mechaniczna (RAMPS 1.4)	Endstop / Limit Switch	6

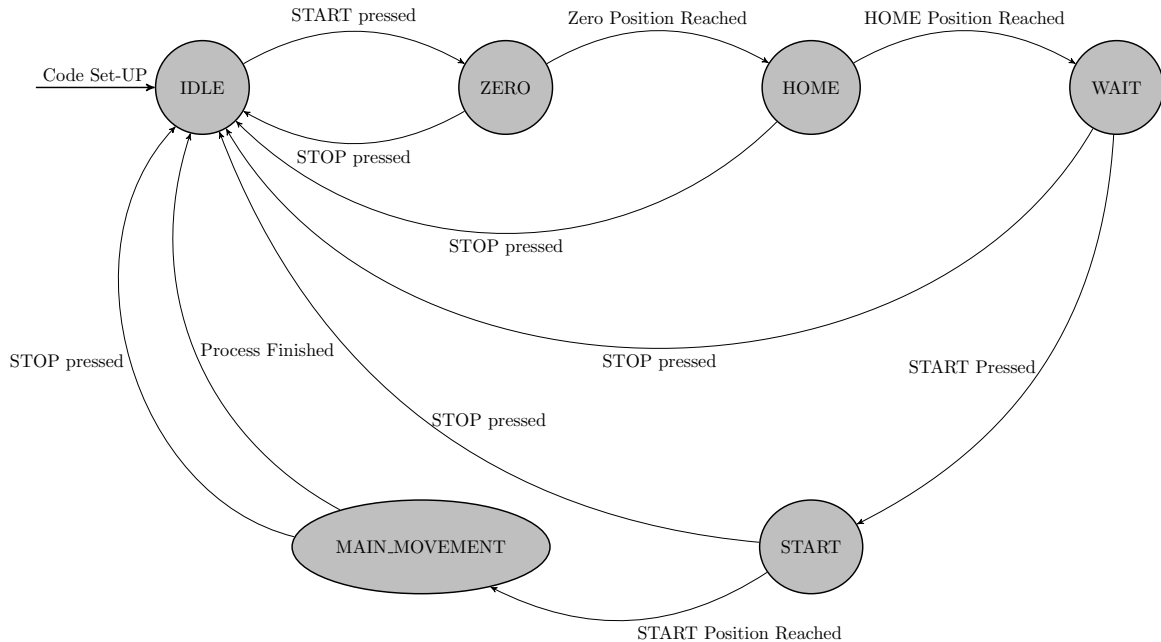
Tabela 2: Zestawienie elementów elektronicznych

7 Oprogramowanie

7.1 Architektura sterowania

W programie zastosowano mechanizm maszyny stanów, który odpowiada za organizację pracy całego układu. Sterowanie zostało podzielone na zestaw jasno zdefiniowanych stanów odpowiadających kolejnym etapom działania mechanizmu, takim jak: oczekiwanie na uruchomienie, procedura bazowania, przejazd do pozycji startowej, oczekiwanie na rozpoczęcie procesu oraz realizacja ruchu roboczego.

Każdy stan posiada własną logikę działania oraz warunki przejścia do kolejnego etapu pracy. Takie rozwiązanie upraszcza zarządzanie algorytmem sterowania, poprawia czytelność kodu oraz ułatwia rozbudowę i diagnostykę systemu.



Rysunek 11: Schemat maszyny stanów

7.2 Konfiguracja sprzętowa układu

Tabela 3: Konfiguracja sterowania silnikami krokowymi

Oś	STEP	DIR
X	GPIO22	GPIO23
Y	GPIO18	GPIO19
Z	GPIO17	GPIO16

Tabela 4: Konfiguracja krańcówek

Krańcówka	Pin	Funkcja
kran_x	GPIO35	Wyznaczanie pozycji zerowej osi X
kran_y	GPIO32	Wyznaczanie pozycji zerowej osi Y
kran_z_1	GPIO33	Dolna krańcówka osi Z
kran_z_2	GPIO25	Górna krańcówka osi Z

Tabela 5: Konfiguracja przycisków sterujących

Przycisk	Pin	Funkcja
START	GPIO14	Rozpoczęcie procedury bazowania lub uruchomienie procesu roboczego
STOP	GPIO12	Natychmiastowe zatrzymanie programu i przejście do stanu IDLE

Tabela 6: Zachowanie diody LED w poszczególnych stanach programu

Stan programu	Zachowanie diody LED
STATE_IDLE	Dioda świeci światłem ciągłym (stan wysoki).
STATE_ZERO	Dioda pozostaje wyłączona po rozpoczęciu procedury bazowania.
STATE_HOME	Dioda zachowuje poprzedni stan (wyłączona).
STATE_WAIT	Dioda miga z okresem 1000 ms, sygnalizując gotowość układu.
STATE_START	Dioda wykonuje trzykrotne mignięcie podczas uruchamiania procesu.
STATE_MAIN_MOVEMENT	Dioda wyłączona podczas wykonywania ruchu roboczego.

7.3 Procedura Homing

Fragment kodu przedstawia implementację procedury Homing zrealizowanej w stanie `STATE_ZERO`. Bazowanie wykonywane jest sekwencyjnie dla osi X, Y oraz Z. Każda oś porusza się do momentu aktywacji odpowiedniej krańcówki, po czym jej aktualna pozycja zostaje ustawiona jako pozycja zerowa przy pomocy funkcji `setCurrentPosition(0)`. Po zakończeniu bazowania wszystkich osi program przechodzi do procedury ustawiania pozycji startowej (`STATE_HOME`).

```
1 case STATE_ZERO:
2
3     if (!is_X_true) {
4         if (value_x == LOW) {
5             stepper1.setCurrentPosition(0);
6             is_X_true = true;
7         } else {
8             stepper1.runSpeed();
9         }
10    }
11
12    if (!is_Y_true) {
13        if (value_y == LOW) {
14            stepper2.setCurrentPosition(0);
15            is_Y_true = true;
16        } else if (is_X_true) {
17            stepper2.runSpeed();
18        }
19    }
20
21    if (!is_Z_true) {
22        if (value_z_1 == LOW) {
23            stepper3.setCurrentPosition(0);
24            is_Z_true = true;
25        } else if (is_Y_true && is_X_true) {
26            stepper3.runSpeed();
27        }
28    }
29
30    if (is_X_true && is_Y_true && is_Z_true) {
31        stepper1.moveTo(-150);
32        stepper2.moveTo(-200);
33        currentState = STATE_HOME;
34    }
35    break;
```

Tabela 7: Przebieg procedury Homing

Etap	Opis działania
1	Ruch osi X do krańcówki zerowej.
2	Wyzerowanie pozycji osi X.
3	Ruch osi Y do krańcówki zerowej.
4	Wyzerowanie pozycji osi Y.
5	Ruch osi Z do dolnej krańcówki.
6	Wyzerowanie pozycji osi Z.
7	Przejsie do ustawiania pozycji startowej HOME.

7.4 Trajektoria robocza

Trajektoria robocza realizowana jest poprzez ciągłą zmianę prędkości osi X oraz Y w funkcji czasu. Algorytm wykorzystuje zależności trygonometryczne funkcji sinus i cosinus, co pozwala uzyskać ruch po trajektorii kołowej.

Prędkości chwilowe osi wyznaczane są według zależności:

$$V_x = -V_{max} \sin(\theta)$$

$$V_y = V_{max} \cos(\theta)$$

gdzie:

- V_x – chwilowa prędkość osi X,
- V_y – chwilowa prędkość osi Y,
- V_{max} – maksymalna prędkość ruchu,
- θ – aktualny kąt ruchu wyznaczany na podstawie czasu trwania cyklu.

Zastosowanie sterowania prędkościowego pozwala na płynne odwzorowanie ruchu kołowego bez konieczności definiowania dyskretnych punktów trajektorii.

Tabela 8: Parametry trajektorii roboczej

Parametr	Wartość	Opis
predkosc_x	150	Maksymalna prędkość osi X [kroki/s]
predkosc_y	150	Maksymalna prędkość osi Y [kroki/s]
kroki_x	100	Promień trajektorii w osi X [kroki]
kroki_y	100	Promień trajektorii w osi Y [kroki]
minuty	1	Czas realizacji procesu [min]
wymagana_liczba	wyliczana	Liczba pełnych cykli trajektorii roboczej

```

1 float obwod_kroki = 2.0 * PI * kroki_x;
2 unsigned long czas_okregu_ms =
3 (obwod_kroki / predkosc_x) * 1000;
4
5 for (int cykl = 0; cykl < wymagana_liczba; cykl++) {
6
7     unsigned long start_cyklu = millis();
8     unsigned long czas_aktualny = millis();
9
10    while (czas_aktualny - start_cyklu < czas_okregu_ms) {
11
12        czas_aktualny = millis();
13
14        float progres =
15        (float)(czas_aktualny - start_cyklu)
16        / czas_okregu_ms;
17
18        float kat = progres * 2.0 * PI;
19
20        float aktualna_predkosc_x =
21        -predkosc_x * sin(kat);
22
23        float aktualna_predkosc_y =
24        predkosc_y * cos(kat);
25
26        stepper1.setSpeed(aktualna_predkosc_x);
27        stepper2.setSpeed(aktualna_predkosc_y);
28
29        stepper1.runSpeed();
30        stepper2.runSpeed();
31    }
32 }
```

7.5 Dodatkowe funkcje programu

7.5.1 Definicja maszyny stanów

Sterowanie programem zrealizowano przy pomocy maszyny stanów FSM. Każdy stan odpowiada za określony etap pracy układu.

```
1 enum State {
2     STATE_IDLE,
3     STATE_ZERO,
4     STATE_HOME,
5     STATE_WAIT,
6     STATE_START,
7     STATE_MAIN_MOVEMENT
8 };
9
10 State currentState = STATE_IDLE;
```

7.5.2 Logika START

Naciśnięcie przycisku START inicjuje procedurę bazowania poprzez wyzerowanie flag stanu oraz przejście do stanu STATE_ZERO.

```
1 case STATE_IDLE:
2
3     if (digitalRead(STARTbutton) == LOW) {
4
5         is_X_true = false;
6         is_Y_true = false;
7         is_Z_true = false;
8
9         stepper1.setSpeed(200);
10        stepper2.setSpeed(200);
11        stepper3.setSpeed(-30);
12
13        currentState = STATE_ZERO;
14    }
15
16    break;
```

7.5.3 Pozycja startowa

Po zakończeniu bazowania mechanizm przejeżdża do zadanej pozycji roboczej HOME.

```
1  stepper1.moveTo(-150);
2  stepper2.moveTo(-200);
3
4  while(stepper1.distanceToGo() != 0 ||
5        stepper2.distanceToGo() != 0){
6
7      stepper1.run();
8      stepper2.run();
9  }
10
11 stepper1.setCurrentPosition(0);
12 stepper2.setCurrentPosition(0);
```

7.5.4 Obsługa STOP

Przycisk STOP monitorowany jest w każdym stanie pracy programu. Jego aktywacja powoduje natychmiastowe przejście do stanu spoczynkowego.

```
1  if (digitalRead(STOPbutton) == LOW)
2      currentState = STATE_IDLE;
```

8 Załączniki